

AI Systems Performance Engineering

Chapter 6 — GPU Architecture, CUDA Programming, and Maximizing Occupancy Part II structure preview (draft)

Tianqin Cai
Applied Scientist
AWS

Internal Training for Applied Scientists on AI Infrastructure

August 15, 2026

Part II — Anatomy of a CUDA Kernel: Device Side

The hardware wants many ready warps — now: how does CUDA code create them?

```
__global__ void myKernel(float* input, int N) {  
    // unique global index for this thread  
    int idx = blockIdx.x * blockDim.x + threadIdx.x;  
    if (idx < N) { // bounds check!  
        input[idx] *= 2.0f; // in-place: double it  
    }  
}
```

- `__global__`: runs on the device, callable from the host.
- Built-ins: `blockIdx` (which block am I), `blockDim` (block size), `threadIdx` (my rank in the block).

- The kernel is written from the perspective of **one thread**; `idx` carves out its slice of the data.
- CUDA compiles it into device code executed by **thousands or millions** of lightweight threads in parallel.
- Launch syntax (host):
`<<<blocksPerGrid, threadsPerBlock>>>` — the two core parameters of every kernel invocation.

Plain English: write the job description for one worker; CUDA prints a million copies, each with a different row number.

Host Side: The Six-Step Data Flow

```
const int N = 1'000'000;
float *h_input = nullptr, *d_input = nullptr; // h_=host, d_=device
cudaMallocHost(&h_input, N * sizeof(float)); // 1) pinned host alloc
for (int i = 0; i < N; ++i) h_input[i] = 1.0f; // init
cudaMalloc(&d_input, N * sizeof(float)); // device alloc
cudaMemcpy(d_input, h_input, N * sizeof(float),
           cudaMemcpyHostToDevice); // 2) H2D copy
const int threadsPerBlock = 256;
const int blocksPerGrid = // = 3,907 for 1M
    (N + threadsPerBlock - 1) / threadsPerBlock;
myKernel<<<blocksPerGrid, threadsPerBlock>>>(d_input, N); // 3) launch
cudaDeviceSynchronize(); // 4) wait for device
cudaMemcpy(h_input, d_input, N * sizeof(float),
           cudaMemcpyDeviceToHost); // 5) D2H copy
cudaFree(d_input); cudaFreeHost(h_input); // 6) cleanup
```

- Naming convention: **h_** = host pointer, **d_** = device pointer — used throughout the book.
- **cudaMallocHost** = **pinned** (page-locked) memory — prerequisite for async copies to truly overlap later.
- Book note: *not fully optimized yet* — this is the simple, complete **template** the rest of the book improves on.

Why Pass N? The Single-Thread Perspective

The book's answer

“A CUDA kernel function is designed to work **inside of a single thread**, alongside thousands of other threads, **on a partition of the input data**. As such, N defines the size of the partition that this particular kernel will process.”

- A CPU function could just inspect the array size. A kernel **cannot** — it sees a raw pointer and must be told where its world ends.
- Combined with blockIdx/blockDim/threadIdx, N lets every element be processed **cleanly and uniquely**, in parallel, across many SMs.
- This is the core mental shift from CPU to GPU programming: you don't write “process the array,” you write “process **my** element — if it exists.”

Plain English: each of the million copied workers gets the same memo; N is the line that says where the job ends.

The Bounds Check — and Lazily Surfacing Errors

Worked example: `myKernel<<<1, 64>>>(input, 63)`

63 elements (0–62), 64 threads = **2 warps**. **Warp 0** (threads 0–31): all valid. **Warp 1** (threads 32–63): **thread 63 is the one extra** beyond 0–62.

- With `if (idx < N)`: lanes 0–30 of warp 1 execute, lane 31 is **masked** (active mask) — one-warp divergence, negligible.
- Without it: `input[63]` is **undefined behavior** — may raise `cudaErrorIllegalAddress`, may **silently pass or corrupt data** (the address may still be **mapped**). OOB bugs are **flaky**. Book rule: no check in sight? **Understand why**.

*Plain English: the GPU doesn't call you when something breaks — it **leaves a note in the mailbox**.*

- Kernels launch **asynchronously**, with **no per-thread exceptions**: a caught fault sets a **global flag** for the launch.

```
myKernel<<<...>>>(...); // error happens HERE
doSomethingOnCPU(); // CPU keeps going
cudaDeviceSynchronize(); // ...but REPORTED here
```

- The sync didn't fail — it **delivered the news**: errors surface **lazily**.
- Debug practice: `cudaGetLastError()` + `cudaDeviceSynchronize()` right after launches.

Configuring Launch Parameters: The Recipe in Code

The recipe (why 256? — justified in Part I's worked example)

`threadsPerBlock = 256` (= 8 warps); `blocksPerGrid = (N + 255) / 256` — integer division rounds **up**, so **every element is covered**.

- The rounding trick generalizes: $(N + B - 1) / B$ for any block size B .
- The **last block may be partially full** — exactly why the **bounds check** (p. 5) exists.
- Book tip for Blackwell: consider **256–512** threads per block.
- Treat 256 as the **starting point**, not the answer: **profile** 128 / 512 variants before committing.

Plain English: 256 is the “medium coffee” of block sizes — almost never the wrong first order; refine later if profiling says so.

2D and 3D Kernel Inputs

```
__global__ void my2DKernel(float* input,
                           int width, int height) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x < width && y < height) { // 2D bounds check
        int idx = y * width + x; // flatten to 1D
        input[idx] *= 2.0f;
    }
}
// host: dim3 threads(16,16); // 256 threads, 2D
// dim3 blocks((W+15)/16, (H+15)/16);
```

- Natural fit for **images**: e.g., a $1,024 \times 1,024$ matrix processed by 16×16 blocks (still 256 threads).
- Same pattern generalizes to 3D with **dim3(x, y, z)** — volumetric data maps directly onto the thread hierarchy.
- Book note: most of the book uses **1D or 2D (tiled)** launches; in 1D, plain ints beat dim3.

Plain English: same recipe, two axes: each worker gets a (row, column) badge instead of a single row number.

Allocate Asynchronously: Streams and Memory Pools

```
cudaStream_t s;  
cudaStreamCreateWithFlags(&s, cudaStreamNonBlocking);
```

```
float* d_buf = nullptr;  
cudaMallocAsync(&d_buf, N * sizeof(float), s);  
myKernel<<<blocks, threads, 0, s>>>(d_buf, N);  
cudaFreeAsync(d_buf, s); // deferred until s finishes
```

- **Free waits only for stream s** — no global `cudaDeviceSynchronize`, no cross-stream sync.

- `cudaMalloc/cudaFree` are **synchronous and expensive**: full-device sync + OS calls (`mmap/ioctl`), kernel-space context switches.
- `cudaMallocAsync/cudaFreeAsync` draw from a per-device **memory pool**: recycled buffers, no OS round trips, **less fragmentation** over long training loops.
- Non-blocking streams avoid legacy **default-stream barriers** (book tip; multistream overlap in Chapter 11).

Plain English: keep a truck fleet in the lot and grab keys — don't buy and return a truck for every delivery.

Tuning the Pool; PyTorch's Caching Allocator

Preview only — Chapter 11 is the deep dive on stream-ordered allocation; pools return in Chapter 12 (CUDA Graphs).

Pool knobs

- `cudaMemPoolAttrReleaseThreshold` (via `cudaMemPoolSetAttribute`): how much reserved memory the pool keeps before releasing to the OS.
- `cudaMemPoolTrimTo`: proactively return memory.
- Trade-off: total **footprint** vs. **fragmentation**.

The PyTorch connection

- PyTorch's **caching allocator** (`PYTORCH_ALLOC_CONF`, formerly `PYTORCH_CUDA_ALLOC_CONF`) is the same idea: reuse GPU memory, avoid a synchronous `cudaMalloc` per new tensor per iteration.

- Verdict from the book: one-off buffers — blocking `cudaMalloc` is fine; **allocation-heavy loops — async + pools** give more consistent performance and higher throughput.